

Système de particules

HAI911I - TP6

15 octobre 2025

1 Introduction

1.1 Objectifs du TP

Lors de ce TP, vous allez manipuler le *Geometry shader* et le *Compute shader* pour générer des quads texturés dans un système de particules. Vous utiliserez la librairie **QtOpenGL**

Vous trouverez sur le Moodle du cours une archive zip contenant une base de code, une application graphique Qt, simulant un système de particules.

Note. Voici plusieurs liens utiles pour le TP (ce n'est pas exhaustif, fouiller sur internet n'est jamais une mauvaise idée) : Learn OpenGL, Documentation Qt GUI.

1.2 Compilation et exécution

L'archive doit compiler telle quelle sur les machines de la salle.

Tapez : `qmake ..` dans un terminal depuis le dossier `build` pour générer le `Makefile`, puis `make` et enfin `./particules` pour exécuter.

Vous pouvez également vous servir de l'IDE Qt Creator, en ouvrant le fichier `.pro`.

1.3 Rendu

Vous déposerez sur Moodle un **court** rapport montrant *des capture d'écrans* de vos résultats, ainsi qu'un *lien vers le dépôt git* de votre code. Dans le cas où un exercice vous pose problèmes, indiquez votre raisonnement, et les problèmes auxquels vous avez fait face.

2 Système de particules

Commencez par vous familiariser avec la base de code. Vous trouverez un *header* `systemwindow.h` et le `systemwindow.cpp` associé, qui implémente l'essentiel de la simulation du système de particules. Notez la structure `Particule` dans `particule.h` qui décrit leur comportement (initialisation, animation).

Le programme génère un ensemble de particules à partir de la position origine, envoyées dans une direction aléatoire avec un z positif (`Particule::init()`).

Une animation est lancée (`Particule::animate()` est appelée de façon périodique) et à chaque pas de temps la composante z de leur vitesse est décrémentée. Pour simuler un rebond sur le sol ($z = 0$), lorsque la composante z de la position devient négative, son z est remis à zéro, sa vitesse est changée avec une atténuation (`speed.z = -0.8 * speed.z`) pour faire repartir la particule vers le haut. Chaque particule a une durée de vie limitée, lorsque son âge dépasse la limite la particule est réinitialisée (appel de `Particule::init()` quand `age >= ageMax`). Les particules re-dessinée à chaque pas de temps.

Vous remarquerez la présence d'un nouveau shader, `particule.geom`. Notez que les sorties du *vertex shader* (out) doivent correspondre au entrée du *geometry shader* (in), même lorsque le *geometry shader* n'affecte pas le rendu, ses entrées et sorties ne peuvent pas avoir le même nom.

2.1 Quads

Utilisez le *geometry shader* pour dessiner un quad faisant face à l'écran à la place de chacune des particules.

2.2 Textures

Vous trouverez la texture `smoke.png` dans l'archive téléchargée. Ajouter des coordonnées de texture à ces quads et plaquer la texture. Faire un `discard` des fragments dont la couleur est transparente (i.e. celle récupérée dans la texture).

2.3 Temps

Modifier l'animation des particules en ajoutant une notion de temps. Utilisez soit les `timers` soit `QTime` pour calculer δt (temps écoulé entre chaque pas d'animation). Utilisez ce δt pour calculer les positions des particules en utilisant la gravité.

3 Physique sur GPU

Qt 5 prends en charge OpenGL 4.3 Core qui incorpore les *compute shaders*. Pour rappel, le *compute shader* permet d'utiliser les ressources du processeur graphique pour réaliser des calculs en parallèle. Vous allez donc devoir implémenter les calculs d'animation des particules dans le *compute shader*.

3.1 Particule

Implémentez la structure de particule dans `particle.comp`. **Attention**, les données doivent être alignées en mémoire. Modifiez votre structure sur CPU si besoin (les deux doivent être équivalentes).

3.2 Calculs

Ajoutez au *compute shader* les calculs de physique implémentée à l'exercice 2.3.

3.3 Buffers

Pour communiquer les informations du CPU vers le GPU, de la même façon qu'avec les *Vertex Buffer Object* ou *Index Buffer Object*, il existe un *Storage Buffer Object* qui permet d'envoyer des données au GPU et de les lire ensuite sur le CPU. L'avantage de ces buffers, c'est qu'ils peuvent également être considéré comme des `GL_ARRAY_BUFFER` pour être dessiné via `glDrawArrays`.

Créez un *Storage Buffer Object* et allouez les données des particules dans la fonction `ParticleSystemWindow::initialize()`.

3.4 Programme de calcul

Vous avez peut-être tenté d'ajouter votre *compute shader* au `program` de `ParticleSystemWindow`. Comme l'indique le message d'erreur qui est envoyé dans ce cas là, le *compute shader* ne peut pas être utilisé dans le même programme que celui qui s'occupe du rendu.

Ajoutez à `ParticleSystemWindow` un `compute_program`, qui sera compilé et lié à côté du programme initial. **Note.** Pensez bien à attacher ce programme avant d'envoyer les données.

3.5 Exécution des calculs

Utilisez la fonction `glDispatchCompute` disponible dans `QOpenGLFunctions_4_3_Core`, dans la fonction `ParticleSystemWindow::render()` pour lancer les calculs. La fonction `glMemoryBarrier()` permet de s'assurer la terminaison de ces derniers avant l'affichage.

4 Bonus

Ajoutez des effets tels que changer la taille et/ou la couleur des quads en fonction de l'âge de la particule, définissez plusieurs types de particules et leur donner des trajectoires et/ou des textures différents. Ajoutez des comportements aux particules : rebonds en utilisant les impulsions.